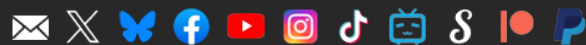


Inigo Quilez :: articles :: ray differentias and texturing - 2015



Motivation

Raymarching distance fields is becoming a popular technique to get procedural graphics rendered in a cheap way (without having to go the marching-cubes or surface-nets route). From fractals to terrains to arbitrarily complex shapes and objects, it is easy to render interesting images with just a few lines of code. However, just like any pixel tracing based technique (raytracing, pathtracing, etc) the technique comes with some drawbacks. One of them is the artifacts one gets when doing surfacing operations that need partial derivatives. For example, texturing objects with the hardware texture mapper in a naive way, which uses `dFdx()` and `dFdy()` to extract texel footprints in screen space, will produce artifacts in the edges of the objects where UV continuity breaks. Any other operation that requires derivatives will suffer too, such as LOD techniques that tweak surface roughness. But let's focus on texturing only here, within the context of a SDF raymarcher:

In the simplest implementation, rendering a distance field by raymarching is comprised of the following steps: constructing a ray passing through a given pixel, the actual raymarching to find an intersection, the normal computation at the intersection point, the shading/surfacing, the lighting and the color correction. The artifact we are discussing in this article happens in the surfacing/texturing step.

The problem is that, unlike in polygon rasterization, when in the classic raymarching setup described above, the texture unit in the GPU is no longer guaranteed to be looking up from the same local surface when we request a mipmap-filtered texture sample. As we know, current GPUs do shading (and hence texturing) in groups of 2x2 pixels. This guarantees that each texture lookup will have neighbors to perform texture-coordinate differences. With those differences the GPU knows many texels will fall into the current pixel, and therefore can infer which mip level is best for sampling the texture without aliasing (ie, having more than two texels per pixel).

Of course, with the raymarching setup we are considering, this pixel differentiation method doesn't work because the whole assumption of surface locality is broken. For example, while one pixel can belong to a tree, the neighboring pixel can belong to a different branch of the tree, or even to a completely different object, such as a rock. This never happens with polygon rasterization because objects are rasterized one at a time and because polygon shading is extended to cover 1 extra pixel in each of the x and y directions when needed

(when the polygon is too small or we are shading its edge). But in our raymarcher this pixel locality isn't guaranteed, the pixel differences no longer measure texture-coordinate derivatives in all cases, and therefore texture filtering is broken at surface (depth) discontinuities. What usually happens is that at those discontinuities the derivatives are overestimated (for there are big differences in texture-coordinates) and texels are brought from the smallest mip level of the texture, creating a visible edge of different color, as seen in the image below.



*The problem of raymarching textured objects, and the proposed solution.
Note the dark pixel at the object silhouettes and in the background*

The solution

The solution is easy: since the hardware cannot compute correct derivatives in all cases, let's do it ourselves. In the case of our classic raymarcher, at the time we are constructing our pixel ray, we can also compute the rays going through the two neighboring pixels (that to the right and that above or below the current one). The difference between the ray direction and these two extra rays are called "ray differentials", and basically they encode the footprint of your pixel in world space (it's size and orientation). Then as we raymarch and traverse the scene this difference becomes larger and proportional to the traveled distance, quite like in a cone tracer. So, as long as we keep these ray differentials around, we'll be able to estimate the footprint of the pixel being

rendered at the intersection point in world space, and then estimate the texture coordinates at the corners of this pixel footprint. That will tell us how many texels we'll need to consider for proper filtering.

The estimation itself can be done by constructing a tangent plane to the surface at the intersection point, by using the point itself (**pos**) and the normal (**nor**) of the surface at the point,

$$\text{plane} = 0$$

and then intersecting the ray of the adjacent pixels into it to estimate the intersection points for them (**posx** and **posy**), which by comparison with the original intersection point **pos** give us an estimation of the area covered by the pixel in texture space.

For perspective camera this reduces to this:

```
vec3 dposdx = t*(rdx*dot(rd,nor)/dot(rdx,nor) - rd);
vec3 dposdy = t*(rdy*dot(rd,nor)/dot(rdy,nor) - rd);
```

and for orthographic it's

```
vec3 dposdx = (ro-rox) - rd*dot(ro-rox,nor)/dot(rd,nor);
vec3 dposdy = (ro-roy) - rd*dot(ro-roy,nor)/dot(rd,nor);
```

Note that in both cases we don't compute the new intersection points explicitly, but the differences with the original point directly, which can avoid floating point precision problems.

Also note that in perspective mode the ray differentials are the ray directions **rd** and for orthographic it's the ray origins **ro**. But this will become clearer with actual code:

Implementation

Let's assume we have the following classic raymarcher, which I have over simplified to setting up a ray for a given pixel, raymarching, surfacing and lighting:

```
void raymarch( out vec4 color, in vec2 pixel )
{
    // setup
    vec3 ro = calcRayOrigin();
    vec3 rd = calcRayDirection( pixel );
```

```

// raymarch: return distance and object id
float (t,oid) = raymarch( ro, rd );

// prepare for surfacing
vec3 pos = ro + t*rd;
vec3 nor = calcNormal( pos, t );

// surfacing
vec2 uv = textureMapping( pos, oid );
vec3 sur = texture( sampler, uv );

// lighting
vec3 lig = calcLighting( pos, nor, t );

// point color
return calcSurfaceColor( sur, lig, t );
}

```

As said, the problem is that both **t** and **oid** (the id of the intersected object) are discontinuous at many pixels (even within the same object). Hence everything downstream in the computations is discontinuous too (**pos**, **nor** and the texture coordinates **uv**).

The proposed solution is to approximate the surface of the object **oid** with a plane tangent to it at the position of the intersection, given the normal that we have. Then, we can take the two rays passing through the neighboring pixels and intersect them analytically with this tangent plane. This is very cheap (free compared to the expensive raymarch we just performed). The difference in the intersection points will give us our pixel world space footprint:

```

void raymarch( out vec4 color, in vec2 pixel )
{
    // setup
    vec3 ro = calcRayOrigin();
    vec3 rd = calcRayDirection( pixel );
    vec3 rdx = calcRayDirection( pixel + vec2(1,0) );
    vec3 rdy = calcRayDirection( pixel + vec2(0,1) );

    // raymarch: return distance and object id
    float (t,oid) = raymarch( ro, rd );

    // prepare for surfacing
    vec3 pos = ro + t*rd;
    vec3 nor = calcNormal( pos, t );

    // compute screen space derivatives of positions analytically for a PERSPECTIVE
    vec3 dposdx = t*(rdx*dot(rd,nor)/dot(rdx,nor)-rd); // vec3 dposdx = dFdx(pos);
}

```

```

vec3 dposdy = t*(rdy*dot(rd,nor)/dot(rdy,nor)-rd); // vec3 dposdy = dFdy(pos);

// surfacing
vec2 (uv,duvdx,duvdy) = textureMapping( pos, dposdx, dposdy, oid );
vec3 sur = textureGrad( sampler, uv, dudvx, dudvy );

// lighting
vec3 lig = calcLighting( pos, nor, t );

// point color
return calcSurfaceColor( sur, lig, t );
}

```

The algorithm hasn't changed a lot. First we compute the neighbor rays (offset by 1 pixel horizontal and vertically). Then we perform the regular raymarch as usual, Then we compute the intersection of the neighbor rays and the planar approximation of the scene at the current intersection point (a plane that passes through it and is tangent to the surface). And lastly we compute the difference in world space positions between these intersections and the actual intersection found for this ray.

Once we have those differences in world space we need to propagate them though the world-space-to-texture-coordinate transform. This can be done with the **Chain Rule**. For planar texture mapping projections this is as simple as a simple linear transform (more of this later).

Lastly, we replace the **texture()** calls for fetching filtered texels from our texture with invocations to **textureGrad()** (this might be called differently depending on the dialect of your shading language in use). This function gets explicit texture-coordinate derivatives ("gradients") and uses them to select the appropriate mip level, rather than resorting to the automatic pixel differencing. The whole point of this technique was of course to reach this **textureGrad()** call prepared with the right information for it to consume.

Details for an orthographic camera

For orthographic views (and isometric), the rays are parallel instead of converging from a single point, so the ray differentials are actually the ray origins, not the ray directions. The code is very similar, but the formulas change as explained at the beginning of the article:

```

void raymarch( out vec4 color, in vec2 pixel )
{
    // setup
    vec3 ro = calcRayOrigin( pixel );
    vec3 rox = calcRayOrigin( pixel + vec2(1,0) );
}

```

```

vec3 roy = calcRayOrigin( pixel + vec2(0,1) );
vec3 rd = calcRayDirection();

// raymarch: return distance and object id
float (t,oid) = raymarch( ro, rd );

// prepare for surfacing
vec3 pos = ro + t*rd;
vec3 nor = calcNormal( pos, t );

// compute screen space derivatives of positions analytically for an ORTHOGRA
vec3 dposdx = (ro-rox) - rd*dot(ro-rox,nor)/dot(rd,nor); // vec3 dposdx = dFd
vec3 dposdy = (ro-roy) - rd*dot(ro-roy,nor)/dot(rd,nor); // vec3 dposdx = dFd

// surfacing
vec2 (uv,duvdx,duvdy) = textureMapping( pos, dposdx, dposdy, oid );
vec3 sur = textureGrad( sampler, uv, dudvx, dudvy );

// lighting
vec3 lig = calcLighting( pos, nor, t );

// point color
return calcSurfaceColor( sur, lig, t );
}

```

Details

I just explained the basic idea but skipped over the "textureMapping()" function. As I said, whatever transform you apply to your world (or object) intersection position in order to deduce texture coordinates, a related transformation will need to be applied to the position derivatives in order to compute the texture coordinate derivatives. A bit of calculus will help you here, and if you are into Jacobians this will be an easy ride for you. But in general the Chain Rule will serve you well. This is a case by case homework to do though, but in the case of a planar projection the results are very easy:

If you were texture mapping your objects with a vertical projection of the form

```
vec2 uv = pos.xz*vec2(0.03,0.07);
```

then the differences in **uv** are

```
vec2 duvdx = dposdx.xz*vec2(0.03,0.07);
```

```
vec2 duvdy = dposdy.xz*vec2(0.03,0.07);
```

If you are using some basic tri-planar mapping (called "rounded box mapping" by some), then you can change your old

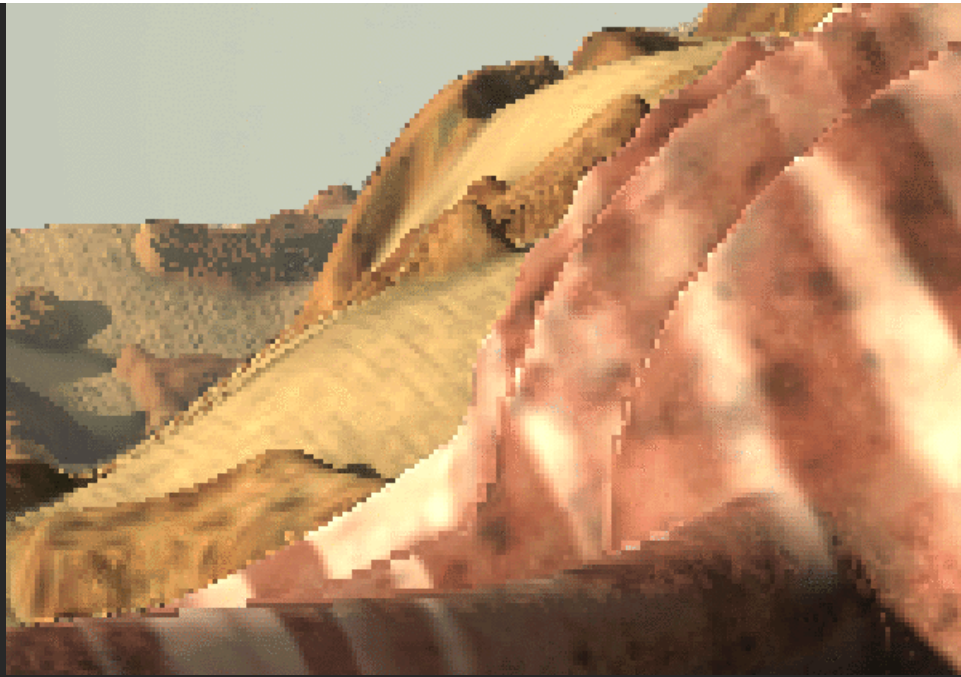
```
vec4 texcube( sampler2D sam, in vec3 p, in vec3 n, in float k, in vec3 gx, in vec3 gy, in vec3 gz )
{
    vec3 m = pow( abs( n ), vec3(k) );
    vec4 x = texture( sam, p.yz );
    vec4 y = texture( sam, p.zx );
    vec4 z = texture( sam, p.xy );
    return (x*m.x + y*m.y + z*m.z) / (m.x + m.y + m.z);
}
```

to

```
vec4 texcube( sampler2D sam, in vec3 p, in vec3 n, in float k, in vec3 gx, in vec3 gy, in vec3 gz )
{
    vec3 m = pow( abs( n ), vec3(k) );
    vec4 x = texture2Grad( sam, p.yz, gx.yz, gy.yz );
    vec4 y = texture2Grad( sam, p.zx, gx.zx, gy.zx );
    vec4 z = texture2Grad( sam, p.xy, gx.xy, gy.xy );
    return (x*m.x + y*m.y + z*m.z) / (m.x + m.y + m.z);
}
```

Here's a link to this technique running live in a raymarcher: <https://www.shadertoy.com/view/4dtGWM>

This is one more comparison of the broken behavior versus the correct one as implemented in this article:



The problem of raymarching textured objects, and the proposed solution

inigo quilez - learning computer graphics since 1994